

4.0 Units: px, %, em, rem, vh, vw, Media queries and breakpoints

4.0.0 Pixel (px)

Definition

- px stands for **pixels**.
- It is an **absolute unit** (fixed size).
- 1px = 1 dot on the screen (depending on screen resolution).

Example

```
p {  
  font-size: 16px;  
}
```

Advantages

- ✓ Predictable and precise.
- ✓ Simple for beginners.

Disadvantages

- ✗ Not scalable → does not adapt to user preferences.
- ✗ Can look too small or too big on different screens.

4.0.1 Percentage (%)

Definition

- A **relative unit**.
- Always calculated **relative to the parent element**.

Example

```
div {  
  width: 50%;  
}
```

Advantages

- ✓ Flexible, automatically adjusts to parent size.

Disadvantages

- ✗ Can be confusing when parent has no fixed size.

4.0.2 em

Definition

- Relative to the **font-size of the parent element**.
- 1em = font-size of the parent.

Example

```
div {  
  font-size: 20px;  
}  
  
p {  
  font-size: 2em; /* = 40px */  
}
```

Nested Example (Multiplication Effect)

```
div { font-size: 16px; }  
section { font-size: 1.5em; } /* 24px */  
p { font-size: 2em; } /* 48px */
```

Advantages

✓ Flexible and scalable.

Disadvantages

✗ Can “multiply” in nested elements → hard to control.

4.0.3 rem (Root em)

Definition

- Relative to the **root (<html>) element’s font-size**.
- 1rem = font-size of <html>.

Example

```
html { font-size: 16px; }  
p { font-size: 2rem; } /* 32px */
```

Advantages

- ✓ Avoids “multiplication problem” of em.
- ✓ Easier to manage responsive typography.

Disadvantages

✗ Less flexible in some nested designs.

4.0.4 vh (Viewport Height)

Definition

- Relative to the **height of the browser’s viewport**.
- 1vh = 1% of viewport height.

Example

```
div {  
  height: 50vh;  
}
```

Advantages

✓ Adapts to screen size automatically.

Disadvantages

✗ May cause issues with mobile browsers when toolbars appear/disappear.

4.0.5 vw (Viewport Width)

Definition

- Relative to the **width of the browser's viewport**.
- 1vw = 1% of viewport width.

Example

```
div {  
  width: 100vw;  
}
```

Advantages

✓ Very useful for **responsive layouts**.

Disadvantages

✗ sometimes overflows if scrollbars are present.

Unit	Type	Relative To	Example	Use Case
px	Absolute	Screen pixels	16px	Borders, icons, fixed design
%	Relative	Parent element	50%	Fluid layouts, grids
em	Relative	Parent font-size	2em	Flexible text, padding
rem	Relative	Root (<html>) font-size	2rem	Consistent typography
vh	Relative	1% of viewport height	50vh	Fullscreen sections
vw	Relative	1% of viewport width	100vw	Responsive widths

4.1 Media Queries & Breakpoints, Mobile-first design

4.1.0 Introduction

Modern websites must adapt to many devices: mobiles, tablets, laptops, and desktops.

Media queries and **breakpoints** are the core tools in CSS for achieving **responsive design**.

4.1.1 What are Media Queries?

- A **media query** is a CSS feature that applies styles **only if certain conditions are true** (like screen width, height, orientation, resolution, etc.).
- It helps in **customizing design** for different devices.

Syntax

```
@media (condition) {  
  /* CSS styles here */  
}
```

Example

```
@media (max-width: 768px)  
{  
  body {  
    background-color: lightblue;  
  }  
}
```

4.1.2 Why Use Media Queries?

1. To make websites **responsive**.
2. To improve **usability** on small devices.
3. To **avoid horizontal scrolling**.

To serve different layouts for **mobile, tablet, and desktop** users.

4.1.3 What are Breakpoints?

- **Breakpoints** are specific **screen widths** where the layout of a webpage changes.
- They are used with **media queries**.

Common Breakpoints (industry standard):

- **320px – Small phones**
- **480px – Large phones**
- **768px – Tablets**
- **1024px – Small laptops**
- **1200px+ – Desktops**

Types of Media Queries

a) **Max-width** (applies up to a certain width)

@media (max-width: 768px)

```
{ p { font-size: 14px; }  
}
```

b) Min-width (applies from a certain width upwards)

@media (min-width: 1024px)

```
{ p { font-size: 18px; }  
}
```

c) Min & Max Combined (specific range)

@media (min-width: 768px) and (max-width: 1023px)

```
{ p { font-size: 16px; }  
}
```

d) Other Conditions

Orientation:

@media (orientation: landscape)

```
{ body { background: lightgreen; }  
}
```

Resolution:

@media (min-resolution: 300dpi)

```
{ img { width: 80%; }  
}
```

4.1.4 Mobile-First vs Desktop-First Approach

a) Mobile-First (Recommended)

Default styles written for **small screens** first.

Use min-width for larger devices.

Example:

```
/* Default (mobile) */
```

```
p { font-size: 14px; }
```

```
/* Tablet and above */
```

```
@media (min-width: 768px)
```

```
{ p { font-size: 16px; } }
```

```
/* Desktop and above */
```

```
@media (min-width: 1024px)
```

```
{ p { font-size: 18px; }
```

```
}
```

b) Desktop-First

- Default styles written for **large screens**.
- Use max-width for smaller devices.

Example:

```
/* Default (desktop) */
```

```
p { font-size: 18px; }
```

```
/* Tablet */
```

```
@media (max-width: 1024px)
```

```
{ p { font-size: 16px; }
```

```
}
```

```
/* Mobile */
```

```
@media (max-width: 768px)
```

```
{ p { font-size: 14px; }
```

```
}
```

4.1.5 Advantages of Media Queries

- ✓ Flexible design for all devices.
- ✓ Better **user experience**.
- ✓ Helps in **SEO ranking** (Google prefers mobile-friendly sites).
- ✓ Saves cost (no need for separate mobile website).

4.1.6 Disadvantages

- ✗ Too many breakpoints = complex CSS.
- ✗ Harder to maintain if not planned properly.

4.1.7 Practical Example – Responsive Box

```
<!DOCTYPE html>

<html>

<head>

<style>

.box {

    width: 100%;

    height: 100px;

    background: green;

}

/* Tablet */

@media (min-width: 768px) {

    .box

    { background:

    blue;

    }

}

/* Desktop */

@media (min-width: 1024px) {

    .box

    { background:

    red;

    }

}

</style>

</head>

<body>

    <div class="box"></div>
```

4.2 Responsive navigation concepts, Practical

4.2.0 Introduction

Navigation is one of the most important parts of a website.

- On **desktops**, space is available for a full horizontal navigation bar.
- On **mobiles**, screen width is limited → so we must design navigation that is **responsive**.
- A **responsive navigation** adapts its layout depending on the device size.

4.2.1 Types of Navigation Layouts

a) Horizontal Navigation (Desktop)

- Links are shown **side by side** in a row.
- Common for large screens.

Example:

```
nav ul
```

```
{ display:  
flex;  
justify-content: space-around;  
}
```

b) Vertical Navigation (Mobile)

- Links are stacked in a **column**.
- Saves space on smaller screens.

Example:

```
nav ul
```

```
{ display:  
flex;  
flex-direction: column;  
}
```

c) Hamburger Menu (☰)

- Popular design pattern for mobile.
- A small icon (☰) represents the menu.
- On click/tap → it expands to show navigation links.

4.2.2 Responsive Navigation Techniques

a) Using Flexbox

- Flexbox allows arranging menu items **horizontally or vertically**.
- Example:

```
nav {  
  display: flex;  
  justify-content: space-between;  
}
```

b) Using Media Queries

- Show/hide navigation based on **breakpoints**.
- Example:

```
/* Mobile */  
.menu { display: none; }  
.hamburger { display: block; }  
/* Tablet & Desktop */  
@media (min-width: 768px) {  
  .menu { display: flex; }  
  .hamburger { display: none; }  
}
```

c) Hide/Show Menu

- Use display: none; and display: block; to toggle menus.
- For mobile → hide menu by default, show on button click.

d) Dropdowns / Sidebars

- On mobile, menus can **slide in from the side**.
- Achieved with **CSS transitions** or **JavaScript toggles**.

4.2.3 Mobile-First Responsive Navigation

a) Default (Mobile)

- Show only logo and hamburger menu (☰).
- Menu hidden by default.

b) Tablet/Desktop

- Show full menu inline.
- Hide hamburger icon.

4.2.4 Practical Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Navigation</title>
  <style>
    body
    { margin:
    0;
      font-family: Arial, sans-serif;
    }
    nav {
      display: flex;
      justify-content: space-between;
      align-items: center;
      background: #333;
      padding: 1rem;
    }
    .logo
    { color:
      #fff;
      font-size: 1.5rem;
    }
    .menu
    { display:
```

```
background: #444;
position: absolute;
top: 60px;
right: 20px;
width: 200px;
}
.menu a { color:
  #fff; padding:
  1em;
  text-decoration: none;
  display: block;
}
.hamburger
  { font-size:
    2rem; color:
    #fff; cursor:
    pointer;
  }
/* Tablet & Desktop */
@media (min-width: 768px) {
  .menu
    { display:
      flex;
      flex-direction: row;
      position: static;
      background: none;
      width: auto;
    }
  .hamburger
    { display: none;
    }
}
```

```
</style>
</head>
<body>
  <nav>
    <div class="logo">MySite</div>
    <div class="hamburger">≡</div>
    <div class="menu">
      <a href="#">Home</a>
      <a href="#">About</a>
      <a href="#">Services</a>
      <a href="#">Contact</a>
    </div>
  </nav>
</body>
</html>
```

4.2.5 Advantages of Responsive Navigation

- ✓ Works across devices (mobile, tablet, desktop).
- ✓ Saves screen space on small devices.
- ✓ Improves user experience.
- ✓ Reduces bounce rate (users stay longer).

4.2.6 Challenges

- ✗ Requires careful design for different screen sizes.
- ✗ Sometimes needs **JavaScript** for toggling hamburger menu.
- ✗ More testing required across devices.

4.3 Deploy using GitHub Pages, Final portfolio polishing

4.3.0 What is GitHub Pages?

Imagine you made a website (HTML + CSS files) in your laptop. Now you want to **show it to the world** — friends, teachers, companies.

- GitHub Pages = free hosting by GitHub.
- You don't need to buy hosting, just upload your files to GitHub → GitHub will give you a live website link like:

<https://yourusername.github.io/>

4.3.1 Folder Structure (very simple)

```
my-portfolio/  
├── index.html  
├── about.html  
├── css/  
│   └── style.css  
├── images/  
│   └── profile.jpg
```

4.3.2 Steps to Deploy (Step by Step)

Step 1 – Create GitHub Repository

1. Go to github.com → Sign in.
2. Click **New Repository** → Name it yourusername.github.io (⚡ this name is special, it makes your main website).
3. Keep it **Public** → Click **Create Repository**.

Step 2 – Upload Code

With Git commands:

- git init
- git add .
- git commit -m "my portfolio first commit"
- git branch -M main

- `git remote add origin https://github.com/username/username.github.io.git`
- `git push -u origin main`

Step 3 – Enable GitHub Pages

1. Go to **Settings** → **Pages**.
2. Under **Source**, choose main branch.
3. Save → Wait 1-2 minutes.
4. Visit <https://yourusername.github.io> → Your site is live!